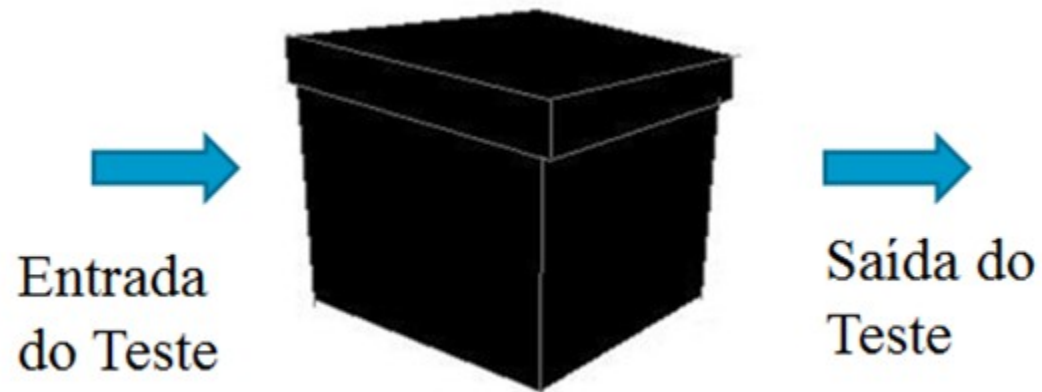




TESTE DE SOFTWARE II

TESTE FUNCIONAL

Prof. Raul Sidnei Wazlawick
UFSC-CTC-INE

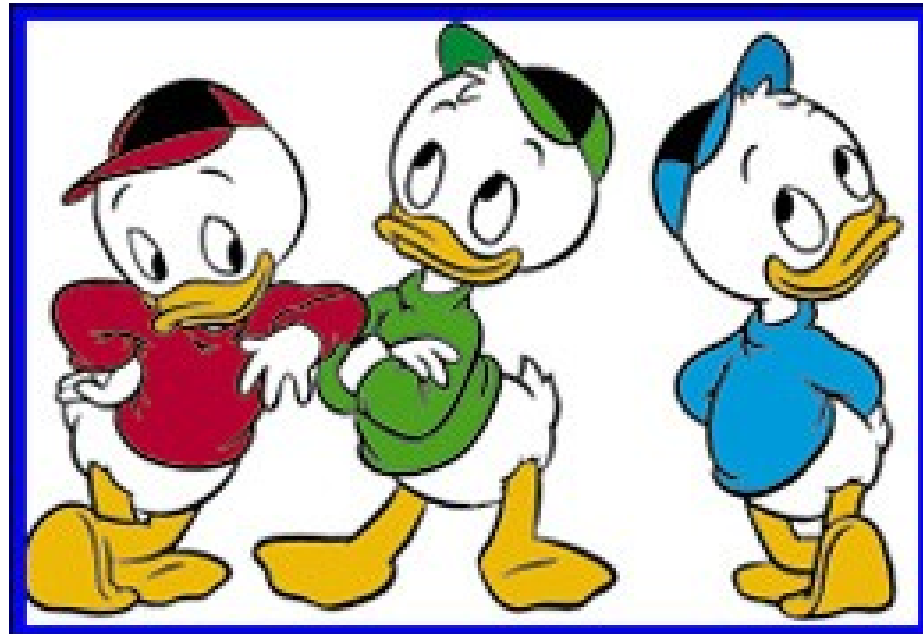
CONTEÚDO

- Teste funcional
 - Particionamento de equivalência
 - Análise de valor limítrofe
 - Execução do teste funcional em nível de sistema
- TDD – Desenvolvimento orientado por testes
- Medição em teste
- Depuração

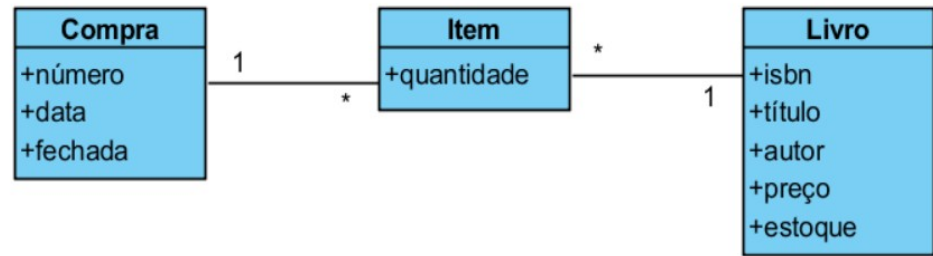


PARTICIONAMENTO DE EQUIVALÊNCIA

- Classes de equivalência



EXEMPLO



- *adicionarItem(idCompra, isbn, quantidade)*
 - Precondições:
 - O parâmetro *idCompra* se refere a uma compra existente e que não esteja fechada.
 - O parâmetro *isbn* se refere a um livro que está cadastrado.
 - O parâmetro *quantidade* é um inteiro positivo (maior do que zero).
 - O parâmetro *quantidade* deve ser menor ou igual ao atributo *estoque* do livro.
 - Pós condições:
 - Se o livro já consta na compra, a quantidade do item referente é incrementada com o valor de *quantidade* e o valor de *estoque* do livro é decrementado no mesmo valor.
 - Se o livro não consta na compra, é criado um item correspondente e sua quantidade definida como *quantidade* e o valor de *estoque* do livro é decrementado no mesmo valor.

CLASSES DE EQUIVALÊNCIA

Parâmetro	Classes válidas	Classes Inválidas
idCompra	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	i1.1: idCompra não corresponde a nenhuma compra existente i1.2: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é verdadeiro
isbn	v2.1: isbn é igual ao atributo isbn de um livro que consta na compra v2.2: isbn é igual ao atributo isbn de um livro que não consta na compra	i2.1: isbn não corresponde a nenhum livro cadastrado
quantidade	v3.1: quantidade é maior do que zero e menor do que o atributo estoque do livro	i3.1: quantidade é menor ou igual a zero i3.2: quantidade é maior do que o atributo estoque do livro

CASOS DE TESTE

Tipo	#	idCompra	isbn	quantidade	Resultado
Sucesso	s1	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	v2.1: isbn é igual ao atributo isbn de um livro que consta na compra	v3.1: quantidade é maior do que zero e menor do que o atributo estoque do livro	O valor da quantidade do item é incrementado e o estoque do livro decrementado no mesmo valor
	s2	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	v2.2: isbn é igual ao atributo isbn de um livro que não consta na compra	v3.1: quantidade é maior do que zero e menor do que o atributo estoque do livro	Foi criado um novo item, com quantidade igual ao parâmetro de mesmo nome e estoque do livro decrementado no mesmo valor
Falha	f1	i1.1: idCompra não corresponde a nenhuma compra existente	v2.2: isbn é igual ao atributo isbn de um livro que não consta na compra	v3.1: quantidade é maior do que zero e menor do que o atributo estoque do livro	Erro de compra inexistente
	f2	i1.2: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é verdadeiro	v2.1: isbn é igual ao atributo isbn de um livro que consta na compra	v3.1: quantidade é maior do que zero e menor do que o atributo estoque do livro	Erro de compra fechada
	f3	i1.2: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é verdadeiro	v2.2: isbn é igual ao atributo isbn de um livro que não consta na compra	v3.1: quantidade é maior do que zero e menor do que o atributo estoque do livro	Erro de compra fechada
	f4	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	i2.1: isbn não corresponde a nenhum livro cadastrado	<i>quantidade</i> pode ser qualquer valor positivo, pois não há estoque para comparar	Erro de livro não cadastrado

CASOS DE TESTE

Tipo	#	idCompra	isbn	quantidade	Resultado
Falha	f5	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	v2.1: isbn é igual ao atributo isbn de um livro que consta na compra	i3.1: quantidade é menor ou igual a zero	Erro de incremento com quantidade não positiva
	f6	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	v2.2: isbn é igual ao atributo isbn de um livro que não consta na compra	i3.1: quantidade é menor ou igual a zero	Erro de adição com quantidade não positiva
	f7	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	v2.1: isbn é igual ao atributo isbn de um livro que consta na compra	i3.2: quantidade é maior do que o atributo estoque do livro	Erro de incremento acima do estoque
	f8	v1.1: idCompra é igual ao atributo número de uma compra existente cujo atributo fechada é falso	v2.2: isbn é igual ao atributo isbn de um livro que não consta na compra	i3.2: quantidade é maior do que o atributo estoque do livro	Erro de adição acima do estoque

TIPOS PRIMITIVOS ESTÁTICOS

- Número naturais
- Números pares
- Números primos
- Datas (dia, mês e ano)
- Valor de ISBN (apenas a formatação, sem levar em conta se o valor foi atribuído a um livro ou não)
- CPF (apenas formatação)
- Intervalo (duas datas, sendo a primeira necessariamente menor ou igual do que a segunda)



CONJUNTOS DINÂMICOS



Os números das compras
já efetuadas.



Os CPFs de pessoas
cadastradas.



As datas que
correspondem ao
aniversário de uma
pessoa cadastrada.

ANÁLISE DE VALOR LIMÍTROFE



EXEMPLO

- Idade (inteiro)
 - Menor que 0: inválido.
 - Entre 0 e 17: válido – menor de idade.
 - Entre 18 e 64: válido – adulto.
 - Entre 65 e 120: válido – idoso.
 - Acima de 121: inválido – idoso improvável.
- Valores a serem testados:
 - -10, -1, 0, 8, 17, 18, 30, 64, 65, 80, 120, 121, 156



TESTE FUNCIONAL EM NÍVEL DE SISTEMA



EXEMPLO

Caso de Uso: Comprar livros

Fluxo principal:

1. O usuário insere palavras-chave para pesquisa de livros.
2. O sistema apresenta a lista de livros (título, autor, preço) que atende às palavras-chave.
3. O usuário seleciona um livro e indica a quantidade desejada.
4. O sistema informa a relação de livros no pedido e o valor total do pedido.
5. O usuário finaliza a compra.
6. O sistema confirma a compra.

Fluxos de tratamento de exceção:

E3a: *O usuário indica uma quantidade acima do estoque.*

E3a.1. O sistema informa a quantidade em estoque do livro em questão.

E3a.2. O usuário informa uma quantidade menor ou igual ao estoque.

Avança ao passo 4 do fluxo principal.

E5a: *O usuário ainda não fez login.*

E5a.1. O usuário faz login informando nome de usuário e senha.

E5a.2. O sistema confirma o login.

Retorna ao passo 5 do fluxo principal.

Fluxos de variantes:

V5a: *O usuário deseja selecionar mais livros na mesma lista.*

Retorna ao passo 3 do fluxo principal.

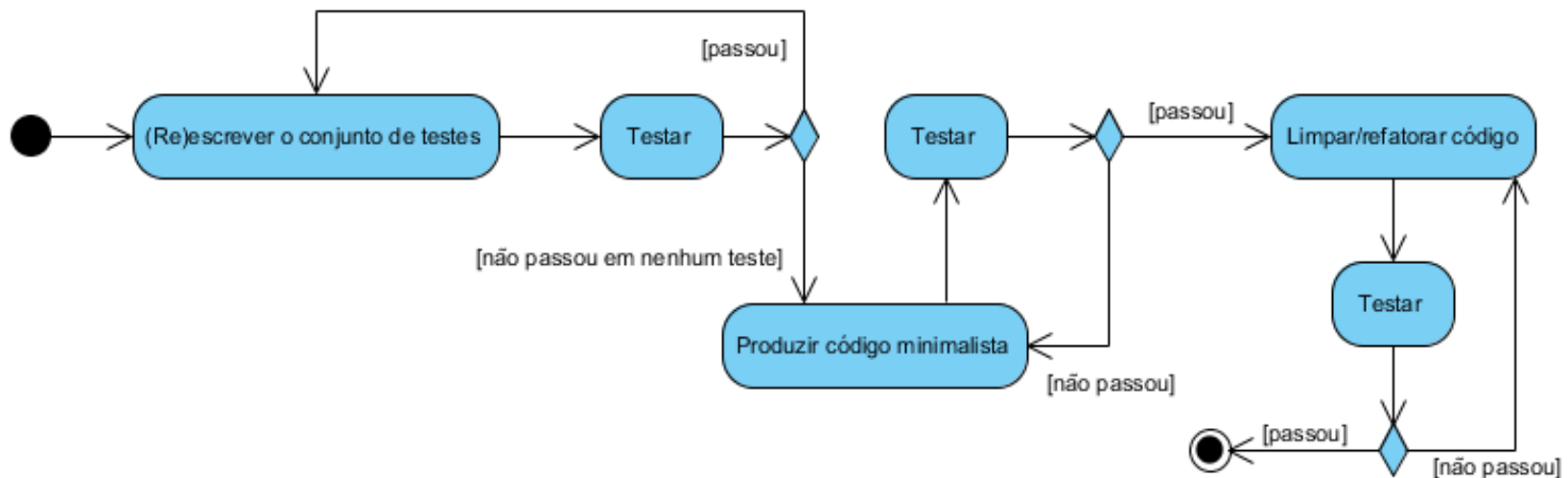
V5b: *O usuário deseja fazer uma nova pesquisa.*

Retorna ao passo 1 do fluxo principal.

CASOS DE TESTE

#	Objetivo	Caminho	Como testar	Resultados
1	Fluxo principal	1, 2, 3, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com o livro e quantidade indicados foi registrada
2	E3a: O usuário indica uma quantidade acima do estoque.	1, 2, 3, E3a.1, E3a.2, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade acima do estoque, é informado da quantidade válida, em seguida indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com o livro e quantidade indicados após a informação sobre estoque foi registrada
3	E5a: O usuário ainda não fez login.	1, 2, 3, 4, 5, E5a.1, E5a.2, 6	Um usuário não logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque e finaliza a compra. É então solicitado a fazer login, faz login e finaliza a compra novamente	Uma compra com o livro e quantidade indicados foi registrada
4	V5a: O usuário deseja selecionar mais livros na mesma lista.	1, 2, 3, 4, 5, 3, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque, em seguida seleciona outro livro e indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com os livros e respectivas quantidades foi registrada
5	V5b: O usuário deseja fazer uma nova pesquisa.	1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque, em seguida faz uma nova pesquisa, seleciona outro livro e indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com os livros e respectivas quantidades foi registrada

TDD - DESENVOLVIMENTO ORIENTADO POR TESTES



EXEMPLO

- categoria(idade)
 - *Menor*, classe válida de 0 a 17 anos.
 - *Adulto*, classe válida a partir de 18 anos.
 - *Não nascido*, classe inválida, até -1 anos.



CÓDIGO INICIAL

```
01 def categoria(idade):
02     return
03
04 #teste de classes válidas usando valor limítrofe
05 menor = 'menor'
06 adulto = 'adulto'
07 assert categoria(0) == menor
08 assert categoria(8) == menor
09 assert categoria(17) == menor
10 assert categoria(18) == adulto
11 assert categoria(45) == adulto
12
13
14 #teste de classes inválidas usando valor limítrofe
15 for idade in [-20, -1]:
16     try:
17         categoria(idade)
18         raise Exception
19     except: pass
```

CÓDIGO MINIMALISTA PARA PASSAR NO TESTE

```
01 def categoria(idade):
02     assert idade >= 0
03     if idade <= 18:
04         return "menor"
05     else:
06         return "adulto"
07     return
08
09 #teste de classes válidas usando valor limítrofe
10 menor = 'menor'
11 adulto = 'adulto'
12 assert categoria(0) == menor
13 assert categoria(8) == menor
14 assert categoria(17) == menor
15 assert categoria(18) == adulto
16 assert categoria(45) == adulto
17
18 #teste de classes inválidas usando valor limítrofe
19 for idade in [-20, -1]:
20     try:
21         categoria(idade)
22     raise Exception
23 except: pass
```

CÓDIGO MINIMALISTA PARA PASSAR NO TESTE

```
01 def categoria(idade):
02     assert idade >= 0
03     if idade < 18:
04         return "menor"
05     else:
06         return "adulto"
07     return
08
09 #teste de classes válidas usando valor limítrofe
10 menor = 'menor'
11 adulto = 'adulto'
12 assert categoria(0) == menor
13 assert categoria(8) == menor
14 assert categoria(17) == menor
15 assert categoria(18) == adulto
16 assert categoria(45) == adulto
17
18 #teste de classes inválidas usando valor limítrofe
19 for idade in [-20, -1]:
20     try:
21         categoria(idade)
22     raise Exception
23 except: pass
```



MUDANÇA NO REQUISITO

- *Menor*, classe válida de 0 a 17 anos.
- *Adulto*, classe válida de 18 a 64 anos.
- *Idoso*, classe válida de 65 a 120 anos.
- *Não nascido*, classe inválida, até -1 anos.
- *Improvável*, classe inválida, a partir de 121 anos.



NOVOS TESTES SÃO ESCRITOS

```
01 def categoria(idade):
02     assert idade >= 0
03     if idade < 18:
04         return "menor"
05     else:
06         return "adulto"
07     return
08
09 #teste de classes válidas usando valor limítrofe
10 menor = 'menor'
11 adulto = 'adulto'
12 idoso = 'idoso'
13 assert categoria(0) == menor
14 assert categoria(8) == menor
15 assert categoria(17) == menor
16 assert categoria(18) == adulto
17 assert categoria(45) == adulto
18 assert categoria(64) == adulto
19  assert categoria(65) == idoso
20 assert categoria(80) == idoso
21 assert categoria(120) == idoso
22
23 #teste de classes inválidas usando valor limítrofe
24 for idade in [-20, -1, 121, 200]:
25     try:
26         categoria(idade)
27         raise Exception
28     except: pass
```

PROGRAMA FINAL

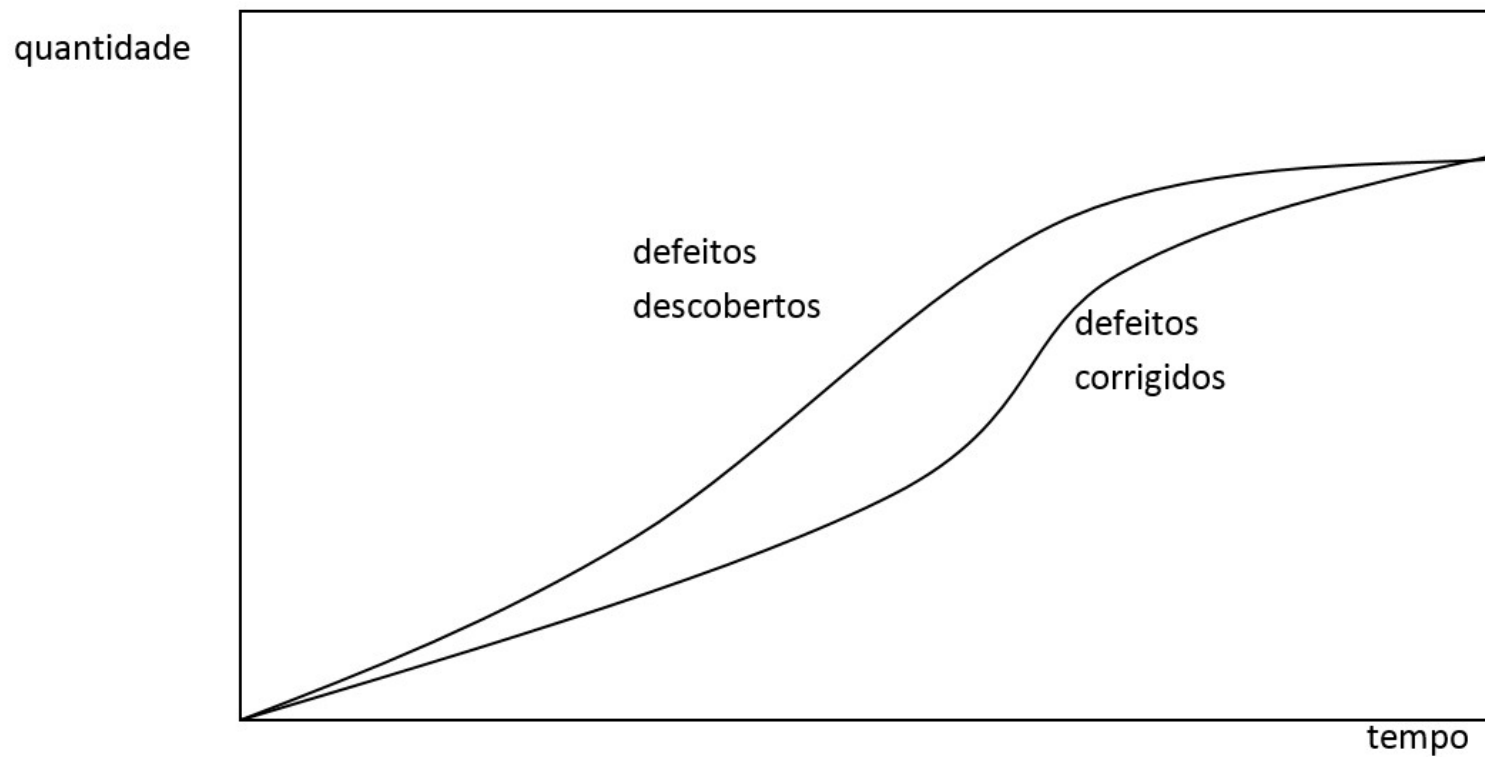
```
01 def categoria(idade):
02     assert idade >= 0
03     assert idade <= 120
04     if idade < 18:
05         return "menor"
06     elif idade < 65:
07         return "adulto"
08     else:
09         return "idoso"
10     return
11
12 #teste de classes válidas usando valor limítrofe
13 menor = 'menor'
14 adulto = 'adulto'
15 idoso = 'idoso'
16 assert categoria(0) == menor
17 assert categoria(8) == menor
18 assert categoria(17) == menor
19 assert categoria(18) == adulto
20 assert categoria(45) == adulto
21 assert categoria(64) == adulto
22 assert categoria(65) == idoso
23 assert categoria(80) == idoso
24 assert categoria(120) == idoso
25
26 #teste de classes inválidas usando valor limítrofe
27 for idade in [-20, -1, 121, 200]:
28     try:
29         categoria(idade)
30         raise Exception
31     except: pass
```

MEDIÇÃO EM TESTE

- Métricas do processo de teste
 - Número de testes previstos (sua necessidade já foi identificada).
 - Número de testes planejados (casos de teste definidos).
 - Número de testes executados, incluindo: (a) testes que falharam e (b) testes que foram aprovados.
- Métricas de teste do produto:
 - Número de defeitos descobertos.
 - Número de defeitos corrigidos.
 - Distribuição dos defeitos por gravidade.
 - Distribuição dos defeitos por classe ou pacote.



EVOLUÇÃO ESPERADA DAS MEDIÇÕES DE TESTE DO PRODUTO



DEPURAÇÃO

